

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



CSC10014 - TƯ DUY TÍNH TOÁN
HKII: 2025 - 2026
**HỆ THỐNG LƯU TRỮ
DU LỊCH THÔNG MINH**

Lớp : CQ2024/6
Nhóm thực hiện : Nhóm 12
Giáo viên hướng dẫn : Hồ Tuấn Thanh
GVHDTH : Mai Anh Tuấn
Trợ giảng : Phạm Nguyễn Sơn Tùng

Thành Viên:

24120033: Đào Tiến Đạt.
24120040: Dương Hoài Đức.
24120304: Trần Thị Bích Hạnh.
24120310: Nguyễn Đình Hiếu.
24120483: Hoàng Văn Trường.

THÀNH PHỐ HỒ CHÍ MINH, NGÀY 18/6/2026

Lời cảm ơn

Trong quá trình thực hiện đề án môn học, nhóm chúng em đã nhận được rất nhiều sự hỗ trợ và hướng dẫn quý báu từ các thầy cô và bạn bè.

Trước hết, chúng em xin gửi lời cảm ơn chân thành đến thầy Hồ Tuấn Thanh – giảng viên phụ trách học phần, đã cung cấp những kiến thức nền tảng cũng như tạo điều kiện để chúng em có cơ hội tiếp cận với các công nghệ và quy trình phát triển ứng dụng thực tế. Chúng em cũng xin chân thành cảm ơn thầy Mai Anh Tuấn – giảng viên hướng dẫn thực hành cùng các thầy cô trợ giảng đã tận tình hỗ trợ, giải đáp các thắc mắc trong quá trình học tập và thực hiện đề án.

Thông qua học phần này, các thành viên trong nhóm không chỉ tích lũy được những kiến thức chuyên môn quý báu mà còn rèn luyện được tư duy giải quyết vấn đề, kỹ năng làm việc nhóm và khả năng vận dụng kiến thức vào các bài toán thực tế. Những kinh nghiệm thu được trong quá trình thực hiện đề tài sẽ là nền tảng quan trọng cho việc học tập và nghiên cứu trong tương lai.

Mặc dù đã nỗ lực hoàn thành đề án với tinh thần học hỏi và trách nhiệm cao nhất, song do kiến thức và kinh nghiệm thực tiễn còn hạn chế nên bài làm khó tránh khỏi những thiếu sót. Chúng em rất mong nhận được những ý kiến đóng góp và nhận xét quý báu từ Thầy/Cô để có thể tiếp tục hoàn thiện kiến thức và kỹ năng của mình.

Cuối cùng, chúng em xin kính chúc quý Thầy/Cô luôn dồi dào sức khỏe, hạnh phúc và gặt hái được nhiều thành công trong sự nghiệp giảng dạy và nghiên cứu.

Các thành viên của nhóm 12

Mục lục

1	Group member	3
2	Idea	4
2.1	Đặt vấn đề	4
2.2	Tầm nhìn sản phẩm	4
2.3	Người dùng mục tiêu	4
3	Problem Analyst & Decomposition	6
3.1	Problem Analyst	6
3.1.1	Đầu vào (Input)	6
3.1.2	Đầu ra (Output)	6
3.1.3	Operators	7
3.1.4	Evaluation Function	7
3.1.5	Constraint	8
3.2	Decomposition	8
3.2.1	Môi trường phát triển và hạ tầng:	9
3.2.2	Cơ sở dữ liệu (Database):	9
3.2.3	Hệ thống Frontend:	9
3.2.4	Hệ thống Backend:	10
3.2.5	Kết luận	10
4	Kiến thức Tư duy Tính toán được áp dụng	10
4.1	Nhận diện mẫu (Pattern Recognition)	11
4.2	Trừu tượng hóa (Abstraction)	11
4.3	Thuật toán (Algorithms)	11
5	System Overview	12
5.1	Tổng quan kiến trúc	12
5.1.1	Kiến trúc:	12
5.1.2	Hệ thống lưu trữ chính	12
5.2	Các Bên Liên Quan Chính	12
5.3	Các Tác Nhân Chính Trong Hệ Thống	13
5.4	Các tính năng cơ bản	13
5.4.1	Tìm Kiếm Accommodations Qua Chatbot AI	14
5.4.2	Chức năng so sánh các địa điểm lưu trú	14
5.4.3	Chức năng trích xuất thông tin chi tiết và đánh giá của AI về một địa điểm	15
5.5	Điểm nổi bật về đổi mới	15
6	Pattern Recognition	16
7	Abstraction	17

8	System/Algorithm Design	18
8.1	Biểu diễn Bài toán	18
8.2	Tổng quan Công nghệ	18
8.3	Mô hình C4	19
8.3.1	Cấp 1 — System Context	19
8.3.2	Cấp 2 — Container	19
8.3.3	Cấp 3 — Component	20
8.3.4	Cấp 4 — Code (Chi tiết Module AI)	20
8.4	Thiết kế Giải thuật theo Chức năng	20
8.4.1	Điều phối AI	20
8.4.2	Tìm kiếm Lai (Hybrid Search)	21
8.4.3	Gợi ý Xếp hạng	22
8.4.4	Đường ống RAG	23
8.4.5	Các Giải thuật Hỗ trợ	23
8.5	Đánh giá Hiệu năng và Khả năng Mở rộng	23
9	Implementation	24
9.1	Từ Mô hình Thiết kế đến Phân rã Module	24
9.2	Di trú Kiến trúc: Microservices → Modular Monolith	24
9.3	Các Thách thức và Giải pháp	24
9.3.1	Tính phi tất định của LLM	24
9.3.2	Streaming SSE qua POST	25
9.3.3	Tích hợp Đa Nhà cung cấp Bản đồ	25
9.3.4	Vector Embedding và Prisma	25
9.3.5	Xác thực Firebase	25
9.4	Các Thách thức Khác	26
9.5	Kết luận	26
10	Testing	27
10.1	Chiến lược Kiểm thử	27
10.2	Phạm vi Kiểm thử	27
10.2.1	Frontend — Giao diện và Tương tác	27
10.2.2	Backend — Xử lý Nghiệp vụ và Tích hợp	27
10.2.3	Các Tình huống và Giả lập Đặc thù	28
10.3	Kết quả Kiểm thử	28
10.4	Cải tiến Sau Kiểm thử	29
10.5	Hướng Phát triển	30
11	Demo	31
12	Deployment	32
13	Logbook: Plan and Actual Workload	33
14	AI usage and declaration	34

15 Kết luận và hướng phát triển	35
15.1 Kết luận	35
15.2 Những kiến thức và kỹ năng đạt được	35
15.3 Hướng phát triển trong tương lai	36

1 Group member

- **Thành viên 1:**
Tên: Đào Tiến Đạt
MSSV: 24120033
Email:
Vai trò/Nhiệm vụ:
- **Thành viên 2:**
Tên: Dương Hoài Đức
MSSV: 24120040
Email:
Vai Trò/Nhiệm vụ:
- **Thành viên 3:**
Tên: Trần Thị Bích Hạnh
MSSV: 24120304
Email: 24120304@student.hcmus.ed.vn
Vai trò/Nhiệm vụ:
- **Thành Viên 4:**
Tên: Nguyễn Đình Hiếu
MSSV: 24120310
Email:
Vai trò/Nhiệm vụ:
- **Thành viên 5:**
Tên: Hoàng Văn Trường
MSSV: 24120483
Email:
Vai trò/Nhiệm vụ:

2 Idea

2.1 Đặt vấn đề

Hiện nay, việc tìm kiếm một nơi lưu trú phù hợp khi đi du lịch không hề đơn giản. Người dùng thường phải cân nhắc nhiều yếu tố như giá cả, vị trí, tiện nghi, chất lượng dịch vụ và các đánh giá từ những khách hàng trước đó. Tuy nhiên, quá trình này vẫn còn gặp nhiều khó khăn do các nguyên nhân sau:

- **Thông tin phân tán trên nhiều nền tảng:** Thông tin về khách sạn, homestay hay resort thường được đăng tải trên nhiều website và ứng dụng khác nhau. Người dùng phải mất nhiều thời gian để tìm kiếm, so sánh giá cả, tiện nghi và đánh giá trước khi đưa ra quyết định.
- **Khó tìm được địa điểm phù hợp với nhu cầu cá nhân:** Mỗi người dùng có những tiêu chí khác nhau về nơi lưu trú như ngân sách, vị trí, loại hình chỗ ở hoặc các tiện ích đi kèm. Tuy nhiên, các công cụ tìm kiếm truyền thống chủ yếu dựa trên bộ lọc cố định nên chưa hỗ trợ tốt cho việc tìm kiếm theo nhu cầu tự nhiên của người dùng.

Hệ quả: Người dùng phải tốn nhiều thời gian để thu thập và xử lý thông tin nhưng vẫn có thể gặp khó khăn trong việc lựa chọn nơi lưu trú phù hợp nhất với nhu cầu của mình.

Giải pháp: Nhóm đề xuất xây dựng một hệ thống tìm kiếm và gợi ý lưu trú thông minh, cho phép người dùng mô tả nhu cầu bằng ngôn ngữ tự nhiên. Hệ thống sẽ phân tích yêu cầu, tìm kiếm các địa điểm phù hợp, thực hiện đánh giá và xếp hạng, sau đó đưa ra những gợi ý trực quan và dễ hiểu nhằm hỗ trợ người dùng đưa ra quyết định nhanh chóng và hiệu quả hơn.

2.2 Tầm nhìn sản phẩm

Tầm nhìn của Smacco là trở thành người bạn đồng hành không thể thiếu trong mỗi chuyến đi, giúp mọi người tìm thấy "ngôi nhà thứ hai" một cách dễ dàng và tin cậy nhất.

Ứng dụng hướng tới việc chuẩn hóa dữ liệu lưu trú đa nguồn, kết hợp với sức mạnh của RAG để cung cấp những hiểu biết sâu sắc từ hàng ngàn ý kiến người dùng. Smacco không chỉ là một công cụ tìm kiếm, mà là một hệ sinh thái nơi công nghệ AI và trải nghiệm con người giao thoa để giải quyết các vấn đề thực tiễn.

2.3 Người dùng mục tiêu

Smacco tập trung phục vụ ba nhóm đối tượng chính:

- **Khách du lịch tự túc:** Những người cần tìm kiếm thông tin chi tiết, so sánh giá và chất lượng một cách nhanh chóng để tối ưu hóa ngân sách và trải nghiệm.
- **Người dùng đang lưu trú (On-site Users):** Những người muốn đóng góp thông tin thực tế, hình ảnh tại chỗ và hỗ trợ cộng đồng thông qua việc trả lời các câu hỏi thực tế.

- **Người dùng cần sự hỗ trợ cá nhân hóa:** Những người ưu tiên việc sử dụng chatbot thông minh để tìm kiếm theo nhu cầu đặc biệt thay vì tự tay lọc qua hàng trăm kết quả.

3 Problem Analyst & Decomposition

3.1 Problem Analyst

Để xây dựng hệ thống Smacco, nhóm đã tiến hành phân tích bài toán và chia nhỏ các yêu cầu thành những thành phần chức năng chính của hệ thống. Việc phân tích này giúp nhóm hiểu rõ luồng xử lý dữ liệu, xác định các chức năng cần thiết và xây dựng giải pháp phù hợp với mục tiêu của dự án.

3.1.1 Đầu vào (Input)

Hệ thống Smacco tiếp nhận dữ liệu từ ba nguồn chính, tạo nên một tập hợp thông tin đa chiều và phi cấu trúc cần được xử lý:

- **Dữ liệu từ người dùng tìm kiếm:**
 - Truy vấn bằng ngôn ngữ tự nhiên (Natural Language Query): Những câu hỏi mang tính định tính như "Tìm khách sạn yên tĩnh, gần biển, giá dưới 1 triệu tại Đà Nẵng".
 - Bộ lọc (Filters) có cấu trúc: Vị trí địa lý (tọa độ, bán kính), loại hình lưu trú (homestay, khách sạn), ngân sách, và các tiện nghi mong muốn (wifi, hồ bơi).
 - Ngữ cảnh hội thoại: Lịch sử các tin nhắn trong phiên chat giúp AI hiểu rõ hơn ý định của người dùng.
- **Dữ liệu từ người dùng On-site và Cộng đồng:**
 - Các bài đánh giá (Reviews) mới nhất, hình ảnh thực tế và các tệp tin đóng góp (Contributions) từ những người đang có mặt tại địa điểm.
 - Các câu hỏi và câu trả lời trong hệ thống Q&A thực tế.
- **Dữ liệu hệ thống và ngoại vi:**
 - Cơ sở dữ liệu địa điểm hiện có (Places database) từ các nguồn Google Places, OpenStreetMap.
 - Kho lưu trữ Vector (Vector Store) chứa các đoạn văn bản (chunks) đã được nhúng (embedded) từ các đánh giá và dữ liệu đóng góp.

3.1.2 Đầu ra (Output)

Mục tiêu cuối cùng của Smacco là chuyển đổi các dữ liệu rời rạc trên thành những kết quả có giá trị sử dụng cao cho người dùng:

- **Danh sách gợi ý tối ưu:** Trả về các địa điểm lưu trú được xếp hạng theo mức độ phù hợp (Scoring) với nhu cầu cá nhân của người dùng, kèm theo metadata đầy đủ (giá, vị trí, rating).
- **Phản hồi AI thông minh (RAG-based Response):** Các câu trả lời từ chatbot không chỉ mang tính tổng quát mà còn trích xuất được những thông tin cụ thể từ hàng ngàn đánh giá (ví dụ: "Người dùng khen bữa sáng ở đây rất ngon nhưng lưu ý tiếng ồn từ công trường gần đó").

- **Trạng thái thực tế tại địa điểm:** Hiển thị số lượng người đang hiện diện, các câu trả lời trực tiếp từ người on-site, giúp người dùng cảm nhận được "linh hồn" và tình trạng thực tế của nơi mình định đến.
- **Dữ liệu chuẩn hóa:** Chuyển đổi các yêu cầu ngôn ngữ tự nhiên thành định dạng JSON có cấu trúc để các hệ thống khác có thể tiêu thụ.

3.1.3 Operators

Quy trình chuyển đổi từ Input sang Output được thực hiện thông qua một chuỗi các thao tác logic và kỹ thuật:

- **Intent Parsing (Xử lý ý định):** Sử dụng LLM để bóc tách câu lệnh của người dùng thành các tham số máy có thể hiểu được (ví dụ: chuyển "khách sạn rẻ ở Hà Nội" thành `{type: "hotel", budget: "low", location: "Hanoi"}`).
- **Vector Retrieval (Truy xuất Vector):** Thực hiện tìm kiếm ngữ nghĩa (Semantic Search) trên *pgvector* để lấy ra các đoạn thông tin (chunks) liên quan nhất đến câu hỏi của người dùng từ kho dữ liệu đánh giá khổng lồ.
- **Scoring Engine (Công cụ tính điểm):** Áp dụng các thuật toán tính toán trọng số để xếp hạng các địa điểm dựa trên sự kết hợp giữa khoảng cách, giá cả, và điểm số đánh giá từ AI.
- **RAG Pipeline (Quy trình sinh câu trả lời có dẫn chứng):** Kết hợp ngữ cảnh đã truy xuất được với sức mạnh sinh ngôn ngữ của LLM để tạo ra câu trả lời cuối cùng, đảm bảo tính xác thực và có nguồn gốc rõ ràng.

3.1.4 Evaluation Function

Dựa theo các tiêu chí sau để đánh giá khả năng giải quyết vấn đề của chương trình:

1. Tiêu chí về người dùng:

Đánh giá dựa trên phản hồi trực tiếp của người dùng, lượng người dùng thường xuyên của ứng dụng, mức độ tương tác của người dùng. Ngoài ra, tiến hành các cuộc khảo sát nhằm tối ưu hoá quá trình tự đánh giá và cải thiện sản phẩm. Các metrics có thể sử dụng:

(a) Rating Score

$$\text{RatingScore} = \frac{\sum \text{rating}}{n}$$

(b) Sentiment Score

$$\text{SentimentScore} = \frac{\text{positive} - \text{negative}}{\text{total}}$$

- #### 2. Tiêu chí về ứng dụng:
- Đánh giá dựa trên tính chính xác và khả năng đưa ra kết quả dựa theo tập dữ liệu đã có. Hơn nữa, các tiêu chí kỹ thuật khác như thời gian phản hồi của hệ thống, khả năng mở rộng và bảo trì, giao diện thân thiện với các

người dùng

Các thông số cụ thể mà chương trình cần phải đạt được:

- Về độ chính xác: $Precision@k \geq 0.5$ sẽ được xem là tạm ổn, kỳ vọng cuối cùng sẽ là từ 0.7 trở lên
- Về thời gian phản hồi của hệ thống: trung bình độ trễ của các module phải đạt là $Latency \leq 700 - 800 (ms)$
- Về khả năng mở rộng: dùng metrics của hệ thống để kiểm tra khả năng chịu tải hiện thời của hệ thống.

3. **Tiêu chí về tính có ích:** Ứng dụng phải giải quyết được ít nhất một vấn đề của xã hội, phải đảm bảo người dùng có thể sử dụng ứng dụng để nâng cao chất lượng cuộc sống

3.1.5 Constraint

Về cấu trúc hệ thống: Sử dụng kiến trúc Monolith module hóa. Toàn bộ backend được xây dựng thành một ứng dụng duy nhất, nhưng vẫn phân chia rõ ràng thành các module chức năng (Core, Recommendation, NLP,...). Mỗi module đảm nhận một nhóm nghiệp vụ riêng, giúp codebase dễ bảo trì, phát triển nhanh và thuận tiện kiểm thử. Khi cần mở rộng quy mô, các module này có thể tách dần thành service độc lập mà không ảnh hưởng đến logic tổng thể.

Về phạm vi: Chương trình trước tiên sẽ tập trung vào phục vụ các người dùng tại Việt Nam, tập dữ liệu cần tìm kiếm sẽ tập trung vào những địa điểm lưu trú ở Việt Nam

Giao tiếp giữa các Service: Sử dụng API Gateway đóng vai trò làm trung gian, kiểm soát giao tiếp

Về phần cứng: Chương trình không quá nặng, yêu cầu chạy mượt ở máy cấu hình trung bình:

- 16+ GB RAM
- CPU có xung nhịp 3.8 GHz
- Dung lượng chương trình khoảng từ 3 - 5 GB (dành cho nhà phát triển)
- Có thể chạy tốt trên máy không có VGA

3.2 Decomposition

Từ mục tiêu lớn của dự án, hệ thống được phân rã thành các hệ thống con (*Systems*) cốt lõi bao gồm:

- Môi trường phát triển và hạ tầng kỹ thuật
- Hệ thống cơ sở dữ liệu (*Database System*)
- Hệ thống giao diện người dùng (*Frontend System*)
- Hệ thống xử lý logic nghiệp vụ (*Backend System*)

Từ cấu trúc các hệ thống lớn trên, tiến hành chia nhỏ thành các module với các phân hệ (*Subsystems*) chức năng chi tiết như sau:

3.2.1 Môi trường phát triển và hạ tầng:

- Sử dụng Docker Compose để triển khai và kiểm thử ứng dụng tại môi trường cục bộ (*Local environment*).
- Sử dụng Git/GitHub phục vụ quy trình quản lý phiên bản mã nguồn (*Version Control*).

3.2.2 Cơ sở dữ liệu (Database):

- Thiết kế và chuẩn hóa lược đồ quan hệ (*Relational Schema*).
- Tích hợp giải pháp Object Storage để lưu trữ các tệp tin vật lý (hình ảnh, tài liệu).

3.2.3 Hệ thống Frontend:

Cài đặt và cấu hình các thành phần giao diện cốt lõi:

- **Trang đăng nhập:**
 - Tính năng đăng nhập xác thực bằng tài khoản Email.
 - Tính năng đăng nhập nhanh thông qua bên thứ ba (Google OAuth).
- **Trang chính (Main Dashboard):**
 - Thanh điều hướng (Navbar).
 - Bản đồ tương tác: Tích hợp và hiển thị thông qua các External API.
 - Hộp thoại trò chuyện trí tuệ nhân tạo (AI Chatbox).
 - Giao diện tìm kiếm theo bộ lọc (*Filter*): Hỗ trợ lọc theo các tiêu chí như giá cả, thời gian lưu trú, đánh giá tổng thể, vị trí, bán kính tìm kiếm; tích hợp ô nhập văn bản tự do (*Textbox ghi chú*) để người dùng tùy biến đặc điểm mong muốn.
- **Thanh bên kết quả tìm kiếm (Search Sidebar):** Giao diện hiển thị danh sách các địa điểm phù hợp kèm thông tin cơ bản (tên, địa chỉ, số sao đánh giá) và tích hợp nút chức năng chỉ đường.
- **Trang thông tin chi tiết địa điểm:**
 - Giao diện nền tảng Hỏi-Đáp (Q&A) tương tác theo mô hình *Reddit-like*.
 - Giao diện cửa sổ Chatbot được cấu hình riêng cho từng địa điểm lưu trú.
- **Trang so sánh các địa điểm lưu trú:**
 - Hiển thị danh sách các địa điểm được hệ thống xếp hạng tối ưu cho người dùng.
 - Tổng hợp và nêu bật các ưu điểm, nhược điểm của từng địa điểm một cách trực quan.
- **Hồ sơ người dùng (User Profile):** Hiển thị lịch sử các bài đánh giá, trạng thái hoạt động trực tuyến, lối tắt tin nhắn nhanh (*Nút chat*),...

3.2.4 Hệ thống Backend:

Phân chia cấu trúc thành các module chức năng độc lập:

- **Core Module:** Trung tâm quản lý dữ liệu và logic nghiệp vụ cơ bản, đóng vai trò xương sống cho toàn bộ hệ thống backend.
 - Quản lý các tiến trình Đăng nhập / Đăng ký / Phân quyền người dùng.
 - Cung cấp các CRUD API tương tác với các thực thể Đánh giá (*Reviews*), Địa điểm (*Places*), và Người dùng (*Users*).
- **Recommendation Module:** Phân hệ chịu trách nhiệm tính toán và gợi ý địa điểm tối ưu.
 - Thực hiện truy vấn dữ liệu thô từ database.
 - Xử lý danh sách ứng viên (*Candidates*): Đánh giá mức độ phù hợp dựa trên bộ lọc cố định; ứng dụng LLM để phân tích ngữ nghĩa phần ghi chú tự do của người dùng.
 - Xếp hạng (*Ranking*): Thực hiện chấm điểm (*AI Scoring*) và sắp xếp thứ tự ưu tiên.
- **NLP Module:** Phân hệ xử lý ngôn ngữ tự nhiên.
 - Thực hiện kết nối và gọi tới các API mô hình ngôn ngữ bên ngoài.
 - **Parser:** Chuyển đổi và chuẩn hóa văn bản ngôn ngữ tự nhiên thành cấu trúc dữ liệu hệ thống.
 - **Communicator:** Phản hồi người dùng đối với các câu hỏi kịch bản cơ bản.
 - **RAG (Retrieval-Augmented Generation):** Thực hiện cơ chế truy xuất thông tin từ cơ sở tri thức để tạo câu trả lời chính xác theo ngữ cảnh.

3.2.5 Kết luận

Chúng em đã chia nhỏ mục tiêu xây dựng một "Hệ thống lưu trữ thông minh" thành các bài toán con có thể quản lý và thực hiện độc lập:

- **Phân rã chức năng:** Tách biệt các luồng công việc như Tìm kiếm (Search), Gợi ý (Recommendation), và Trò chuyện (AI Chat). Mỗi bài toán con này lại được chia nhỏ thành các module kỹ thuật như *UsersModule*, *PlacesModule*, *RagModule*, v.v.
- **Phân rã luồng dữ liệu:** Các Use Case được bóc tách thành các bước tuần tự (Sequence Diagrams), từ việc xác thực người dùng, trích xuất ý định từ LLM, truy vấn cơ sở dữ liệu cho đến việc chuẩn hóa kết quả trả về.

4 Kiến thức Tư duy Tính toán được áp dụng

Trong quá trình phân tích và xây dựng hệ thống Smacco, chúng em đã vận dụng linh hoạt bốn trụ cột cốt lõi của Tư duy Tính toán để giải quyết các vấn đề phức tạp một cách có hệ thống:

4.1 Nhận diện mẫu (Pattern Recognition)

Việc nhận diện các quy luật lặp đi lặp lại giúp hệ thống xử lý dữ liệu hiệu quả hơn:

- **Chuẩn hóa yêu cầu:** Nhận diện các mẫu tìm kiếm phổ biến của người dùng (về ngân sách, vị trí, tiện nghi) để xây dựng cấu trúc JSON Schema thống nhất, giúp AI có thể trích xuất thông tin (intent parsing) một cách chính xác.
- **Đồng bộ hóa dữ liệu đa nguồn:** Nhận diện các điểm chung trong cấu trúc dữ liệu địa điểm từ nhiều nguồn khác nhau (Google Places, OpenStreetMap) để thiết kế một mô hình dữ liệu (Database Schema) bền vững và có khả năng mở rộng.

4.2 Trừu tượng hóa (Abstraction)

Chúng tôi đã ẩn đi các chi tiết kỹ thuật phức tạp để tạo ra các giao diện lập trình đơn giản và dễ bảo trì:

- **Trừu tượng hóa AI/LLM:** Người phát triển chỉ cần gọi qua các service của *AiModule* mà không cần quan tâm đến cách thức kết nối API hay xử lý lỗi của các nhà cung cấp LLM cụ thể (Groq, OpenAI).
- **Module hóa kiến trúc:** Sử dụng kiến trúc Monolith Module-based để đóng gói logic nghiệp vụ. Mỗi module hoạt động như một "hộp đen", giao tiếp với nhau qua các interface xác định, giúp giảm sự phụ thuộc lẫn nhau (coupling).
- **ORM (Object-Relational Mapping):** Sử dụng Prisma để trừu tượng hóa các câu lệnh SQL phức tạp, giúp việc thao tác với cơ sở dữ liệu trở nên trực quan và an toàn hơn.

4.3 Thuật toán (Algorithms)

Các thuật toán cụ thể được thiết kế để chuyển đổi dữ liệu đầu vào thành giá trị cho người dùng:

- **Thuật toán Xếp hạng (Scoring Algorithm):** Kết hợp nhiều trọng số như khoảng cách địa lý, điểm đánh giá trung bình và mức giá để đưa ra danh sách gợi ý tối ưu nhất.
- **Tìm kiếm Vector (Semantic Search):** Sử dụng thuật toán so khớp tương đồng vector (Cosine Similarity) trong *pgvector* để thực hiện kỹ thuật RAG, giúp chatbot tìm đúng đoạn thông tin liên quan nhất từ hàng ngàn đánh giá của người dùng.
- **Đánh giá hệ thống:** Sử dụng các thuật toán định lượng như *Precision@k* và *Sentiment Analysis* để đo lường hiệu quả và chất lượng của các đề xuất mà hệ thống đưa ra.

5 System Overview

SMACCO tập trung xây dựng một ứng dụng web với giao diện trực quan, tích hợp bản đồ số và trợ lý ảo (chatbot). Điểm nhấn của hệ thống là khả năng tương tác linh hoạt: người dùng có thể giao tiếp tự nhiên với chatbot để AI tự động xử lý và cập nhật kết quả trực tiếp lên giao diện (AI-driven UI), hoặc chủ động sử dụng các công cụ tìm kiếm, lọc và gợi ý thủ công được cung cấp sẵn trên nền tảng. Ngoài ra, hệ thống còn sử dụng đánh giá của người dùng để tăng cường chất lượng gợi ý

5.1 Tổng quan kiến trúc

5.1.1 Kiến trúc:

Được xây dựng theo kiến trúc **Monolithic**, trong đó các chức năng được tổ chức thành các module bên trong cùng một ứng dụng thay vì triển khai thành nhiều dịch vụ độc lập như mô hình Microservices.

- **Frontend:** React, cung cấp giao diện tương tác với người dùng.
- **Backend:** NestJS, chịu trách nhiệm xử lý các nghiệp vụ chính của hệ thống như quản lý người dùng, địa điểm, đánh giá và tìm kiếm.
- **AI Module:** Tích hợp trong hệ thống Backend để hỗ trợ chatbot và các chức năng trí tuệ nhân tạo.
- **Recommendation Module:** Thực hiện chức năng gợi ý địa điểm và cá nhân hóa nội dung cho người dùng.
- **Database:** PostgreSQL dùng để lưu trữ dữ liệu người dùng, địa điểm, đánh giá và lịch sử tương tác.

5.1.2 Hệ thống lưu trữ chính

- PostgreSQL (pgvector extension) + Prisma: lưu trữ chính
- Lưu trữ phía người dùng: Firebase Auth/Firestore
- Sử dụng object storage cho ảnh (ảnh của các places, ảnh được users đăng tải lên reviews)
- Dùng Redis lưu trữ TTL/in-memory store cho conversation (AI Service) (*đang được xem xét*)

5.2 Các Bên Liên Quan Chính

1. Nhà Phát Triển / Chủ Sở Hữu Sản Phẩm

- Quản lý kiến trúc hệ thống, mở rộng chức năng và tích hợp nền tảng
- Sử dụng API và mô hình dữ liệu để phát triển các tính năng mới

2. Người Dùng Cuối (Khách Du Lịch)

- Tìm kiếm accommodations phù hợp
- Tương tác với AI chatbot để tìm hiểu thông tin chi tiết
- Đánh giá và chia sẻ kinh nghiệm

3. Nhà Cung Cấp Dữ Liệu / Người Đóng Góp

- Cung cấp thông tin về accommodations
- Tải lên tài liệu và dữ liệu bổ sung
- Cải thiện chất lượng dữ liệu của hệ thống

4. Các Nhà Cung Cấp Dịch Vụ Ngoài

- Google Maps, OpenStreetMap: cung cấp dữ liệu địa lý
- Firebase: quản lý xác thực
- Groq: cung cấp mô hình LLM

5.3 Các Tác Nhân Chính Trong Hệ Thống

Hệ thống phân rã người dùng thành các vai trò cụ thể dựa trên hành vi tương tác với nền tảng:

- **Traveler (Khách du lịch):** Tác nhân thực hiện duyệt danh sách chỗ lưu trú, đặt câu hỏi, đọc đánh giá và lựa chọn địa điểm phù hợp.
- **On-site Guest (Khách lưu trú tại chỗ):** Tác nhân đã check-in hoặc đang có mặt tại địa điểm; có quyền tham gia vào các cuộc trò chuyện nội bộ và khai thác dữ liệu từ chatbot để lấy thông tin tại chỗ.
- **Reviewer (Người đánh giá):** Tác nhân đóng góp nội dung, để lại các đánh giá giúp làm giàu tập dữ liệu cho hệ thống gợi ý và tối ưu phản hồi của chatbot RAG.
- **Recommendation Seeker (Người tìm kiếm gợi ý):** Vai trò người dùng yêu cầu hệ thống trả về các đề xuất lưu trú cá nhân hóa dựa trên sở thích và tín hiệu ngữ cảnh.
- **Product Owner / Developer (Nhà phát triển):** Tác nhân kỹ thuật sử dụng kiến trúc hệ thống, các cổng API và mô hình dữ liệu để bảo trì, mở rộng hoặc tích hợp nền tảng.

5.4 Các tính năng cơ bản

SMACCO tích hợp 3 tính năng chính để cung cấp trải nghiệm tìm kiếm accommodation thông minh, xã hội và được cá nhân hóa:

5.4.1 Tìm Kiếm Accommodations Qua Chatbot AI

- **Ngữ cảnh sử dụng (Use Case):** Khách du lịch có nhu cầu tìm kiếm chỗ lưu trú nhưng không muốn thao tác thủ công với các bộ lọc phức tạp, hoặc có những yêu cầu tinh tế khó diễn đạt bằng các tiêu chí lọc truyền thống
Ví dụ: Tìm nơi "yên tĩnh để làm việc" hoặc "phù hợp cho gia đình có trẻ nhỏ".
- **Đầu vào (Input):** Câu lệnh hoặc yêu cầu bằng ngôn ngữ tự nhiên từ người dùng thông qua giao diện chat (ví dụ: "Tìm giúp mình một homestay ở Đà Lạt có view rừng thông, giá dưới 1.5 triệu và cho phép mang theo thú cưng").
- **Đầu ra (Output):** Danh sách các địa điểm phù hợp nhất được hiển thị trực tiếp trong cửa sổ chat, kèm theo lời giải thích ngắn gọn từ AI về lý do tại sao các địa điểm này được đề xuất dựa trên yêu cầu của người dùng.
- **Cách thức hoạt động (Operation / Workflow):**
Hệ thống nhận văn bản
→ *AiModule* sử dụng LLM để thực hiện *Intent Parsing*, trích xuất các tham số (vị trí, giá, loại hình, tiện nghi)
→ Các tham số này được gửi đến *RecommendationsModule* để truy vấn và xếp hạng địa điểm
→ Kết quả được trả về chatbot để AI tổng hợp thành câu trả lời tự nhiên.
- **Lợi ích (Benefits):** Tiết kiệm thời gian, cá nhân hóa trải nghiệm tìm kiếm, thấu hiểu nhu cầu thực tế của người dùng thông qua hội thoại tự nhiên.

5.4.2 Chức năng so sánh các địa điểm lưu trú

- **Ngữ cảnh sử dụng (Use Case):** Sau khi tìm kiếm, người dùng thường phân vân giữa 2-3 lựa chọn hàng đầu. Tính năng này giúp họ đặt các địa điểm này lên "bàn cân" để thấy rõ sự khác biệt về mọi mặt trước khi đưa ra quyết định đặt phòng cuối cùng.
- **Đầu vào (Input):** Danh sách ID của các địa điểm (từ 2 đến 4 địa điểm) mà người dùng đã chọn để so sánh từ danh sách kết quả tìm kiếm hoặc danh sách yêu thích.
- **Đầu ra (Output):** Một bảng so sánh trực quan các tiêu chí cứng (giá, hạng sao, khoảng cách, số lượng người ở) và đặc biệt là phần tóm tắt so sánh định tính từ AI (ví dụ: "Chỗ A có view đẹp hơn nhưng chỗ B có dịch vụ khách hàng được đánh giá cao hơn").
- **Cách thức hoạt động (Operation / Workflow):**
Hệ thống tổng hợp dữ liệu chi tiết của các địa điểm được chọn
→ Sử dụng LLM để phân tích toàn bộ các bài đánh giá gần đây của các địa điểm này
→ Trích xuất các điểm mạnh/yếu tương quan
→ Hiển thị bảng so sánh tổng hợp kết hợp dữ liệu cấu trúc và nhận xét AI.
- **Lợi ích (Benefits):** Hỗ trợ ra quyết định nhanh chóng, khách quan; giảm thiểu rủi ro chọn sai địa điểm do thiếu thông tin đối chiếu.

5.4.3 Chức năng trích xuất thông tin chi tiết và đánh giá của AI về một địa điểm

- **Ngữ cảnh sử dụng (Use Case):** Người dùng muốn tìm hiểu sâu về một địa điểm cụ thể mà không có thời gian để đọc hàng trăm bài đánh giá cũ/mới. Họ cần một cái nhìn tổng quan, trung thực và được cập nhật mới nhất về chất lượng dịch vụ thực tế.
- **Đầu vào (Input):** Một địa điểm cụ thể được chọn; yêu cầu trích xuất thông tin hoặc các câu hỏi chuyên sâu về địa điểm đó (ví dụ: "Chất lượng wifi ở đây thế nào?" hoặc "Bữa sáng có thực sự đa dạng không?").
- **Đầu ra (Output):** Bản tóm tắt thông minh gồm các ý chính: Ưu điểm nổi bật, Nhược điểm cần lưu ý, và các Insight thực tế từ người dùng on-site, kèm theo điểm số đánh giá độ tin cậy của thông tin.
- **Cách thức hoạt động (Operation / Workflow):**
Sử dụng kỹ thuật RAG (*Retrieval-Augmented Generation*)
→ Truy xuất các *chunks* văn bản liên quan nhất từ Vector DB (chứa reviews và đóng góp mới)
→ Đưa ngữ cảnh này vào LLM để tóm tắt và đánh giá theo thời gian thực
→ Trả về kết quả cô đọng cho người dùng.
- **Lợi ích (Benefits):** Cung cấp thông tin có chiều sâu và độ tin cậy cao; giúp người dùng nắm bắt nhanh chóng "vấn đề" của địa điểm mà không bị nhiễu bởi các quảng cáo hay review rác.

5.5 Điểm nổi bật về đổi mới

1. Cho phép người dùng tương tác với nhau

Hệ thống không chỉ hỗ trợ tìm kiếm địa điểm mà còn tạo môi trường để người dùng trao đổi, học hỏi lẫn nhau. Mỗi địa điểm đều có nền tảng hỏi đáp (QA) nơi người dùng có thể đặt câu hỏi, trả lời, bình luận, chia sẻ kinh nghiệm thực tế về địa điểm đó. Ngoài ra, hệ thống còn tích hợp tính năng chat trực tiếp giữa các người dùng, giúp họ dễ dàng kết nối, trao đổi thông tin, thảo luận về các địa điểm quan tâm hoặc hỗ trợ nhau trong quá trình lựa chọn.

2. Tích hợp AI phân tích ngôn ngữ tự nhiên và tự động hoá workflow

- Người dùng chỉ cần nhập yêu cầu bằng tiếng Việt tự nhiên, AI sẽ tự động phân tích, chuẩn hoá và chuyển thành tiêu chí tìm kiếm hoặc thực hiện các tác vụ phức tạp như so sánh, gợi ý, tổng hợp đánh giá, thu thập thông tin mới nhất.
- Không chỉ dừng ở tìm kiếm, AI còn tự động hóa các quy trình xử lý, giúp tiết kiệm thời gian và nâng cao trải nghiệm.

6 Pattern Recognition

7 Abstraction

8 System/Algorithm Design

8.1 Biểu diễn Bài toán

Khái niệm: Biểu diễn bài toán là quá trình chuyển đổi bài toán thực tế (xây dựng nền tảng khám phá chỗ ở thông minh) thành mô hình, cấu trúc dữ liệu và ký hiệu mà máy tính có thể xử lý.

Cách nhóm áp dụng 5 dạng biểu diễn:

1. **Biểu diễn dữ liệu:** Các thực thể thực tế (người dùng, địa điểm, đánh giá, hội thoại, tệp tin) được ánh xạ thành 13 mô hình quan hệ trong PostgreSQL. Mỗi thực thể được gán kiểu dữ liệu tương ứng: UUID cho định danh, JSONB cho dữ liệu linh hoạt, vector cho nhúng ngữ nghĩa.
2. **Biểu diễn cấu trúc:** Mỗi quan hệ giữa người dùng—địa điểm—nội dung được mô hình hóa qua ERD với khóa ngoại và chỉ mục. Sự phụ thuộc giữa các module được biểu diễn bằng sơ đồ lớp và C4 Model.
3. **Biểu diễn quy trình:** Luồng tương tác (tìm kiếm → gợi ý → trò chuyện AI → check-in) được mô hình hóa qua sơ đồ tuần tự và flowchart.
4. **Biểu diễn toán học:** Công thức tính điểm gợi ý đa yếu tố (rating, ngân sách, khoảng cách, tiện ích, cận kề) với trọng số và hàm chuẩn hóa.
5. **Biểu diễn logic:** Quy tắc định tuyến ý định người dùng (IF-THEN) qua LLM JSON Mode; logic kiểm tra điều kiện trong workflow registry.

Ba nguyên tắc cốt lõi được tuân thủ:

- **Trừu tượng hóa:** Chỉ lưu nội bộ địa điểm người dùng tương tác, ủy thác khám phá cho API ngoài — loại bỏ chi tiết không cần thiết.
- **Mô hình hóa có cấu trúc:** Mỗi module có ranh giới rõ ràng (controller/service/DTO), dữ liệu và quan hệ được đồng bộ qua Prisma schema.
- **Đa biểu diễn:** Cùng một bài toán (tìm kiếm chỗ ở) được biểu diễn đồng thời dưới dạng flowchart (luồng), công thức (điểm số) và mã giả (giải thuật hợp nhất).

8.2 Tổng quan Công nghệ

Hệ thống sử dụng ngăn xếp công nghệ thống nhất:

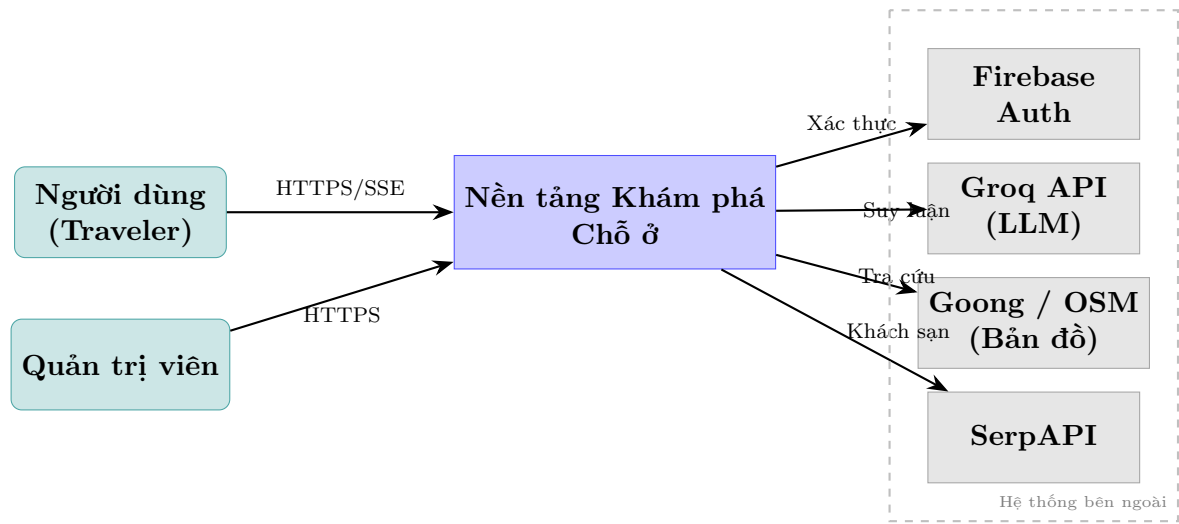
Tầng	Công nghệ	Mục đích
Giao diện Xác thực	React 18 + Vite 7 + TailwindCSS 3.4 Firebase Auth 10 (Web) + Admin SDK 12	SPA, bản đồ Leaflet, responsive Đăng nhập, JWT token
Backend ORM	NestJS 10 + TypeScript 5 Prisma 7.6	API REST, SSE streaming Truy xuất dữ liệu, migrations
Cơ sở dữ liệu	PostgreSQL 16 + pgvector	Dữ liệu quan hệ + vector nhúng
AI	Groq API (llama-3.1-70b)	Định tuyến ý định, tổng hợp phản hồi
Hạ tầng	Docker Compose V2 + Nginx 1.25	Triển khai, reverse proxy

Tích hợp bên ngoài: Goong API (tìm kiếm địa điểm chính), OpenStreetMap/Nominatim (mã hóa địa lý dự phòng), SerpAPI (truy vấn khách sạn).

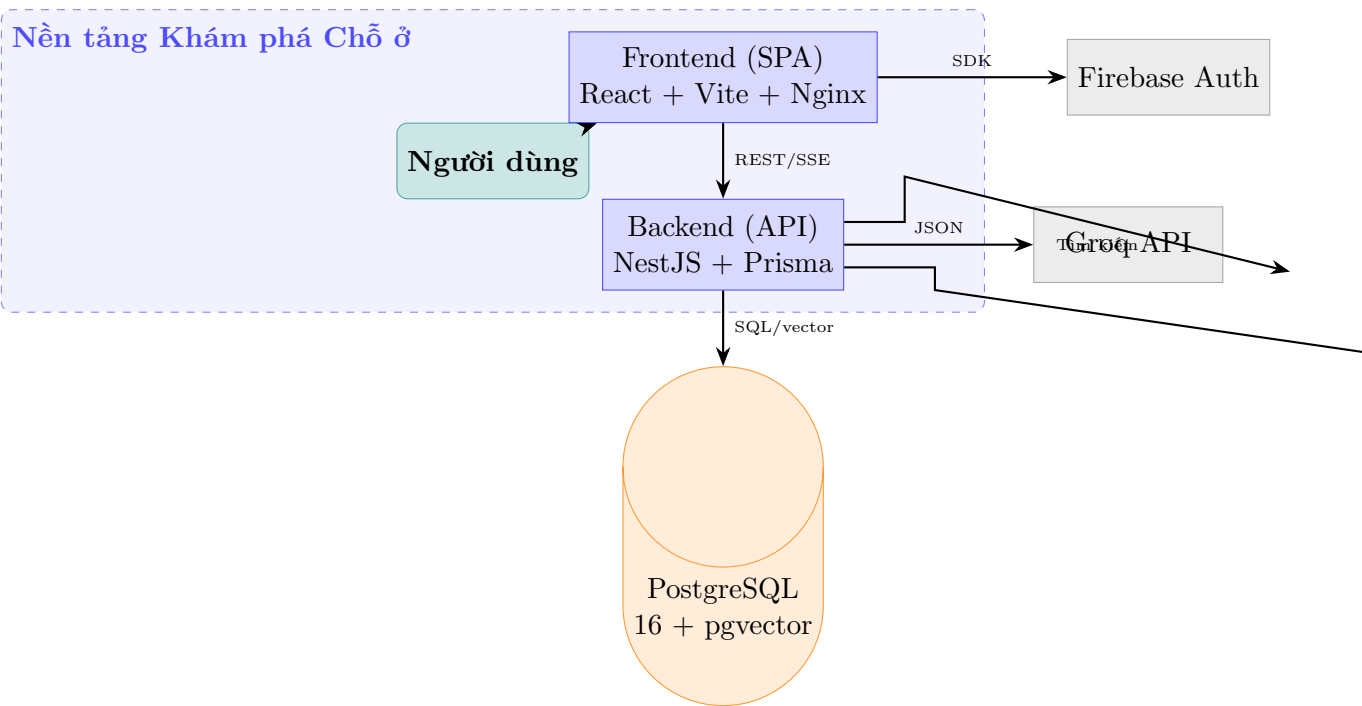
8.3 Mô hình C4

Mô hình C4 được sử dụng để biểu diễn kiến trúc hệ thống ở bốn cấp độ, mỗi cấp độ phục vụ một đối tượng người xem khác nhau.

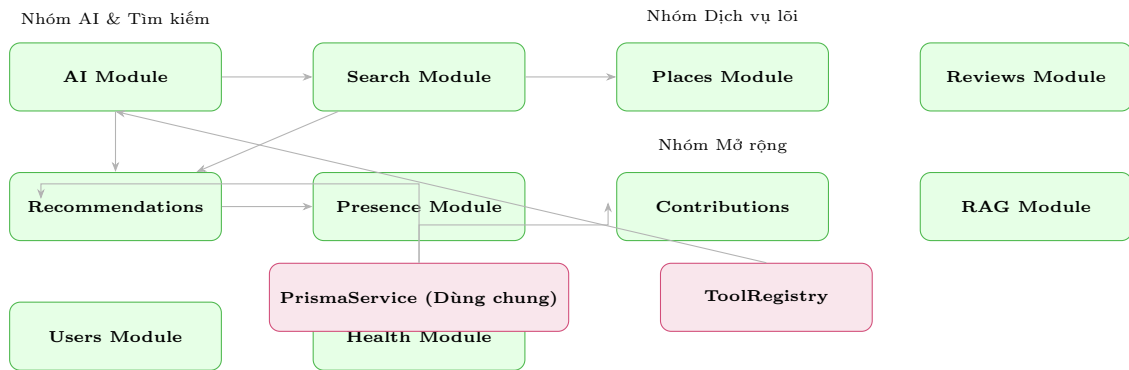
8.3.1 Cấp 1 — System Context



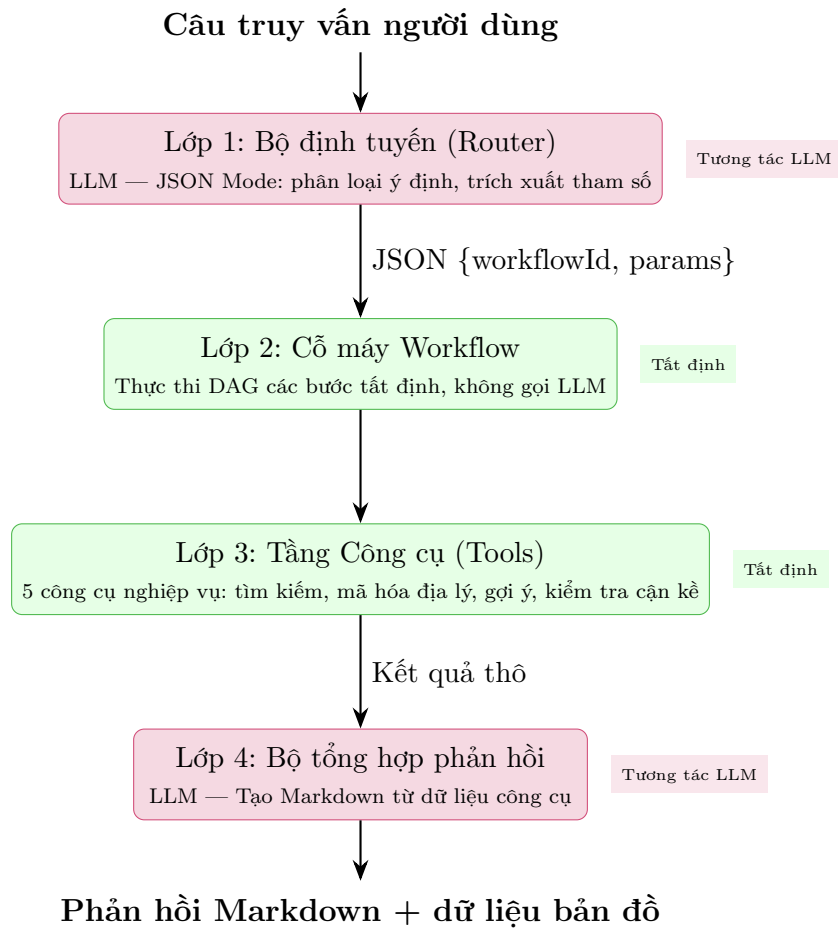
8.3.2 Cấp 2 — Container



8.3.3 Cấp 3 — Component



8.3.4 Cấp 4 — Code (Chi tiết Module AI)

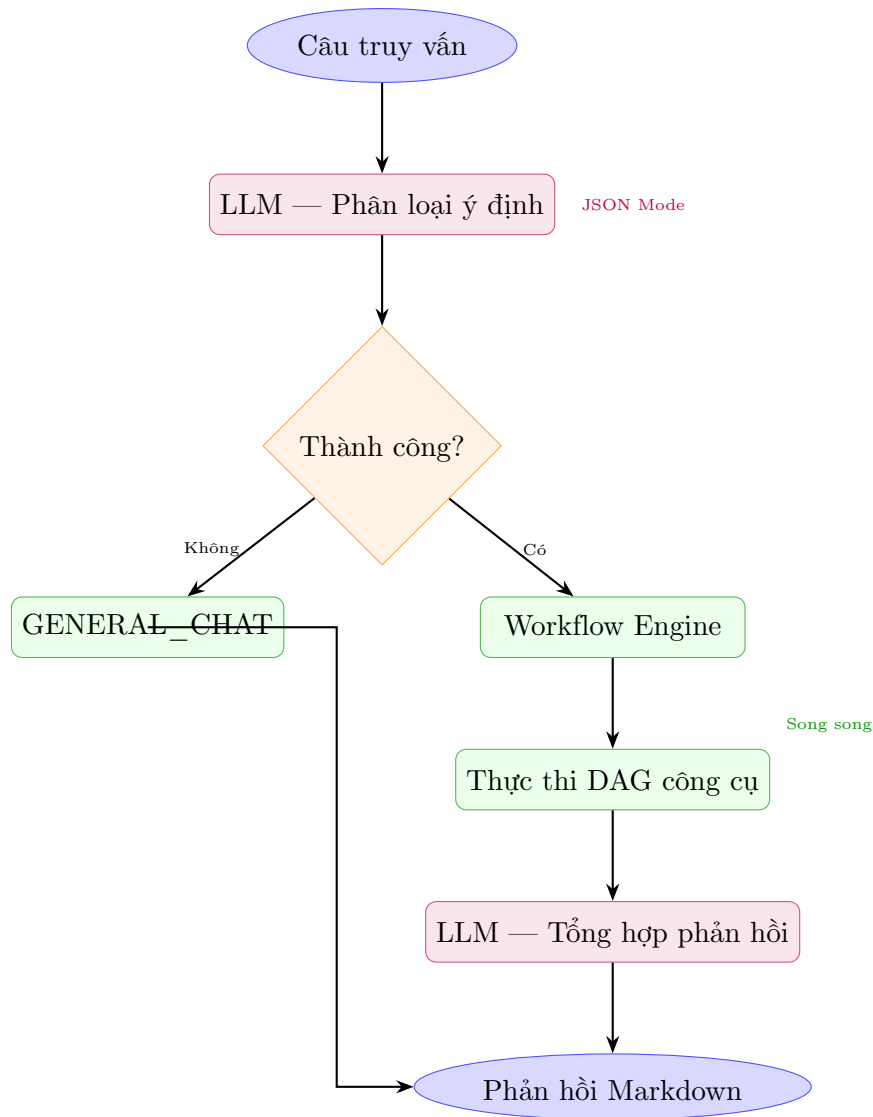


8.4 Thiết kế Giải thuật theo Chức năng

8.4.1 Điều phối AI

Mô tả: Giải thuật điều phối AI chuyển đổi câu truy vấn người dùng thành phản hồi có cấu trúc thông qua kiến trúc bốn lớp.

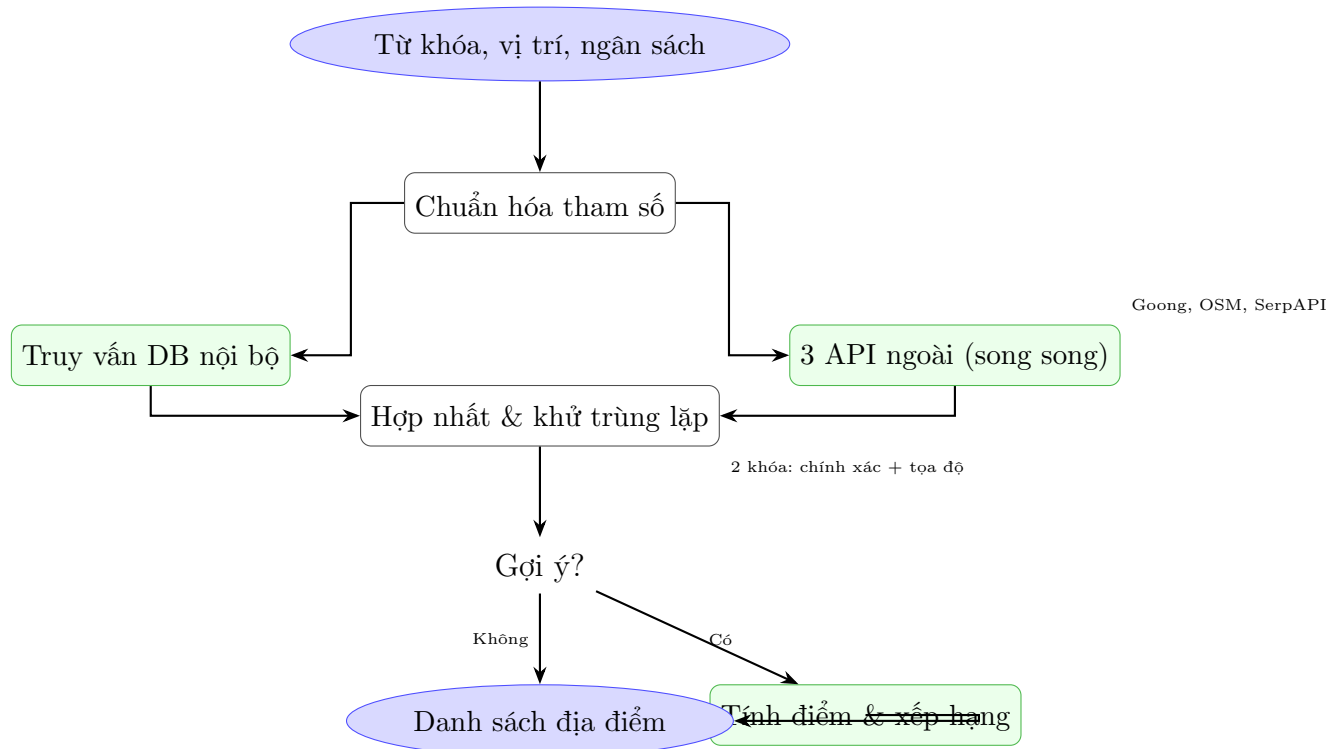
Đầu vào: Câu truy vấn tiếng Việt, lịch sử hội thoại. **Đầu ra:** Phản hồi Markdown kèm dữ liệu bản đồ (searchAction).



Đánh giá: Độ phức tạp $O(|DAG| + |LLM|)$ với $|DAG| \leq 3$. Hai trong bốn lớp không tương tác LLM, đảm bảo tính tất định cho logic nghiệp vụ. Cơ chế an toàn: Router thất bại $\rightarrow$ GENERAL_CHAT; Tool lỗi $\rightarrow$ đánh dấu và tiếp tục.

8.4.2 Tìm kiếm Lai (Hybrid Search)

Mô tả: Kết hợp dữ liệu từ cơ sở dữ liệu nội bộ và ba nhà cung cấp ngoài (Goong, OSM, SerpAPI) để tạo danh sách địa điểm toàn diện.



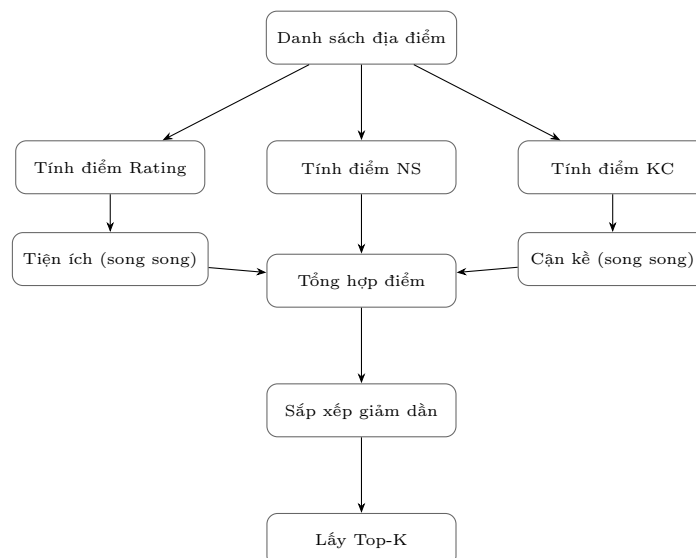
Đánh giá: Độ phức tạp $O(n + m)$. Cách ly lỗi qua catch riêng biệt — một nhà cung cấp thất bại không ảnh hưởng đến các nhà cung cấp khác. Chuỗi dự phòng: Goong → OSM → mảng rỗng.

8.4.3 Gợi ý Xếp hạng

Mô tả: Chuyển đổi kết quả tìm kiếm thô thành danh sách xếp hạng sử dụng mô hình đa yếu tố.

Công thức tính điểm:

$$\text{Điểm} = 0,30 \times \text{Rating} + 0,20 \times \text{Ngân_sách} + 0,20 \times \text{Khoảng_cách} + 0,15 \times \text{Tiện_ích} + 0,15 \times \text{Cận_kề}$$



Khi không có vị trí neo, trọng số khoảng cách và cận kề được phân phối lại cho các yếu tố còn lại. Địa điểm có người đang hiện diện nhận điểm thưởng.

8.4.4 Đường ống RAG

Pipeline phục vụ tìm kiếm ngữ nghĩa trên dữ liệu đóng góp (đánh giá, tệp tin):

- **Tiếp nhận:** Nguồn (review/file) → Phân đoạn văn bản → Sinh embedding → Lưu vào bảng **chunks** với vector.
- **Truy xuất:** Truy vấn → Sinh embedding → Tìm kiếm tương đồng cosine (pgvector) → Xây dựng ngữ cảnh → LLM.

Trạng thái hiện tại: Lược đồ và lưu trữ chunk đã hoàn thiện; sinh embedding đang chờ triển khai.

8.4.5 Các Giải thuật Bổ trợ

- **Theo dõi hiện diện (Presence):** Check-in/check-out với 3 endpoint. Dữ liệu hiện diện phục vụ UI thời gian thực và làm yếu tố tăng điểm trong gợi ý.
- **Bỏ phiếu Q&A:** Mô hình Reddit với ràng buộc UNIQUE(answer_id, user_id). Giá trị vote (1/-1) được vật chất hóa trên bảng answers.
- **Khử trùng lặp:** Chiến lược hai khóa (chính xác: source:source_place_id; mờ: tên_chuẩn_hóa:lat(3):lng(3)) đảm bảo toàn vẹn khi hợp nhất đa nguồn.

8.5 Đánh giá Hiệu năng và Khả năng Mở rộng

Nút thắt	Mức rủi ro	Biện pháp giảm thiểu
Độ trễ LLM (700ms+)	Cao	Streaming, thực thi công cụ song song trước
Giới hạn tốc độ API ngoài	Trung bình	Chuỗi dự phòng, cache
Tìm kiếm vector tập lớn	Trung bình	Chỉ mục IVFFlat
Truy vấn N+1 Prisma	Thấp	Eager loading

9 Implementation

9.1 Từ Mô hình Thiết kế đến Phân rã Module

Từ các mô hình đã xây dựng ở phần Thiết kế Hệ thống (C4 Model, ERD, sơ đồ tuần tự), nhóm tiến hành phân rã thành các module triển khai theo ba bước:

1. **Phân rã chức năng:** Các khối chức năng trong C4 Container → 10 module NestJS, mỗi module gồm tầng giao tiếp (API), tầng logic (xử lý nghiệp vụ) và tầng dữ liệu (DTO).
2. **Ánh xạ dữ liệu:** Mô hình ERD → 13 lược đồ Prisma với quan hệ, ràng buộc và chỉ mục. Các thao tác vector nhúng (pgvector) được xử lý qua SQL thô do hạn chế của ORM.
3. **Ánh xạ quy trình:** Sơ đồ tuần tự → workflow registry dạng JSON và các lớp điều phối AI. Mỗi ca sử dụng tương ứng với một workflow cụ thể trong registry.

Các module được tổ chức thành bốn nhóm:

Nhóm	Module	Nguồn gốc
Dịch vụ lõi	Users, Places, Reviews, Search, Health	Kế thừa từ core-service (NestJS)
AI & Tìm kiếm	AI, Recommendations	Chuyển đổi từ FastAPI sang NestJS
Mở rộng MVP	RAG, Presence, Contributions	Phát triển mới
Dùng chung	PrismaService, ToolRegistry, Global Filters	Toàn cục, dùng chung qua DI

9.2 Di trú Kiến trúc: Microservices → Modular Monolith

Hệ thống nguyên bản vận hành với bốn dịch vụ riêng rẽ (Nginx Gateway, Core-service, AI-service, Recommendation-service). Kiến trúc này bộc lộ các vấn đề:

- **Chi phí mạng:** Mỗi yêu cầu chat trải qua ba chặng HTTP, gây độ trễ tích lũy.
- **Trùng lặp logic:** Xác thực, xử lý lỗi, cấu hình bị sao chép giữa các dịch vụ.
- **Phân mảnh kiểu:** Python ↔ TypeScript yêu cầu tuần tự hóa thủ công, dễ sai sót.
- **Phức tạp vận hành:** Bốn Docker image, bốn pipeline triển khai riêng biệt.

Giải pháp: Hợp nhất thành một monolith dạng module NestJS duy nhất. Các lời gọi HTTP liên dịch vụ được thay thế bằng phương thức gọi trực tiếp qua dependency injection, loại bỏ Gateway Nginx trung gian.

9.3 Các Thách thức và Giải pháp

9.3.1 Tính phi tất định của LLM

Thách thức: LLM sinh phản hồi không nhất quán về định dạng, dễ hallucination dữ liệu, và độ trễ khó dự đoán.

Giải pháp — Kiến trúc Bốn Lớp:

- **Lớp 1 (Router):** LLM với JSON Mode → phân loại ý định, trích xuất tham số có cấu trúc. Thất bại → mặc định GENERAL_CHAT.
- **Lớp 2 (Workflow Engine):** Thực thi DAG các bước tất định, không gọi LLM.
- **Lớp 3 (Tool Layer):** Mỗi công cụ có hợp đồng đầu ra chuẩn hóa, không ném ngoại lệ.
- **Lớp 4 (Composer):** Neo phản hồi Markdown vào dữ liệu thực tế từ công cụ, giảm hallucination.

Lợi ích: Workflow mới có thể thêm dưới dạng JSON, không cần biên dịch lại.

9.3.2 Streaming SSE qua POST

Thách thức: SSE chuẩn chỉ hỗ trợ GET, trong khi endpoint chat cần gửi JSON payload và token xác thực.

Giải pháp — Mô hình hai pha:

- Sử dụng `fetch()` với `ReadableStream` thay vì `EventSource`, cho phép POST và đính kèm token.
- Pha 1: Gửi dữ liệu cấu trúc (`searchAction`) ngay sau khi công cụ thực thi — cho phép frontend render bản đồ trước.
- Pha 2: Truyền token LLM dưới dạng delta SSE.
- Header `X-Accel-Buffering`: no để vô hiệu hóa bộ đệm Nginx khi triển khai sau reverse proxy.

9.3.3 Tích hợp Đa Nhà cung cấp Bản đồ

Thách thức: Mỗi API bản đồ trả về định dạng khác nhau; lỗi một nhà cung cấp không được ảnh hưởng toàn bộ tìm kiếm.

Giải pháp:

- Chuẩn hóa giao diện đầu ra cho mọi nhà cung cấp.
- Gọi song song qua `Promise.all` với `.catch()` riêng biệt — đảm bảo cách ly lỗi.
- Chuỗi dự phòng: Goong → OSM → mảng rỗng.

9.3.4 Vector Embedding và Prisma

Thách thức: Prisma chưa hỗ trợ nguyên sinh kiểu `vector` của PostgreSQL.

Giải pháp: Khai báo cột `embedding` dưới dạng `Unsupported("vector")`, thao tác qua SQL thô (`$queryRawUnsafe`).

9.3.5 Xác thực Firebase

Thách thức: Tích hợp Firebase JWT vào vòng đời NestJS guard mà không sao chép logic xác thực trên mọi controller.

Giải pháp: Triển khai guard với `CanActivate`: trích xuất `Authorization` header → xác thực token qua Firebase Admin SDK → gắn decoded token vào request. Frontend Axios interceptor tự động đính kèm token.

9.4 Các Thách thức Khác

Xử lý đồng thời: Ba mẫu thiết kế được áp dụng: (1) `Promise.all` với catch riêng biệt; (2) ghi vào khóa khác nhau trong shared state để tránh race condition; (3) Active Flag pattern để tránh cập nhật state sau khi component unmount.

Triển khai Docker: Ba container (frontend/Nginx, backend/Node 22, PostgreSQL/pgvector) với chiến lược build đa tầng tận dụng layer cache. Docker override cho môi trường phát triển sử dụng volume ẩn danh tránh xung đột `node_modules`.

Quản lý bản đồ frontend: Chuyển đổi layer bằng opacity, MarkerClusterGroup với chunkedLoading cho hiệu năng, CustomEvent thay vì prop drilling cho tích hợp AI-Search.

9.5 Kết luận

Quá trình triển khai cho thấy việc chuyển đổi từ mô hình thiết kế sang mã nguồn đòi hỏi liên tục giải quyết các vấn đề thực tế: tính phi tất định của LLM, hạn chế của ORM với kiểu dữ liệu đặc thù, và đảm bảo tính nhất quán xuyên suốt các tầng. Mười hai vấn đề đã được ghi nhận với các mức độ ưu tiên khác nhau.

10 Testing

10.1 Chiến lược Kiểm thử

Nhóm áp dụng chiến lược kiểm thử đa tầng, bao phủ từ đơn vị đến tích hợp, với mục tiêu đảm bảo chất lượng cho toàn bộ hệ thống:



Nguyên tắc: Mock toàn bộ tầng dữ liệu và dịch vụ ngoài ở unit test; kiểm thử tích hợp sử dụng module thật với dependency injection đầy đủ. Mỗi tầng kiểm thử bổ sung và mở rộng cho tầng trước.

10.2 Phạm vi Kiểm thử

10.2.1 Frontend — Giao diện và Tương tác

- **Dịch vụ (Service layer):** Kiểm thử 12 dịch vụ frontend bao gồm API chat, tìm kiếm, địa điểm, đánh giá, hiển diện, đóng góp tệp, gợi ý, RAG, người dùng — xác nhận mỗi service gọi đúng endpoint, xử lý đúng dữ liệu trả về và bắt lỗi HTTP.
- **Context:** Kiểm thử 3 context (Auth, TravelData, Theme) — xác nhận provider render đúng, cung cấp giá trị mặc định và cập nhật state khi có thay đổi.
- **Hooks:** Kiểm thử hook streaming AI — ghép chunk delta SSE, xây dựng prompt ngữ cảnh, xử lý lỗi luồng. Kiểm thử ánh xạ lỗi Firebase — dịch 4 mã lỗi (permission-denied, unauthenticated, network-request-failed, unknown) sang thông báo tiếng Việt.
- **Component:** Kiểm thử 25 component giao diện chính: MapComponent (hiển thị marker, cluster), ChatPanel (gửi/nhận tin nhắn, streaming), PlaceCard (hiển thị thông tin, nút tương tác), SearchBar (tìm kiếm, gợi ý), các component wizard-inputs (LocationPicker, BudgetSelector, TypeSelector) và chat subcomponents (MessageList, StreamingText).
- **Trang:** Kiểm thử 5 trang: HomePage, SearchPage, PlaceDetailPage, ProfilePage, AdminPage — xác nhận layout, điều hướng và tích hợp component.

10.2.2 Backend — Xử lý Nghiệp vụ và Tích hợp

- **Dịch vụ (Services):** Kiểm thử 17 service backend bao phủ toàn bộ module: Người dùng (CRUD, upsert, tìm kiếm), Địa điểm (CRUD, lọc, fuzzy match), Đánh giá

(CRUD, điểm số), Tìm kiếm (đa nhà cung cấp, hợp nhất, khử trùng lặp), Gợi ý (tính điểm, xếp hạng, trọng số), Hiện diện (join/leave/get, tải đồng thời), Đóng góp (tạo/lấy/cập nhật tệp), RAG (lưu/truy xuất chunk, xây dựng ngữ cảnh), Hội thoại AI (danh sách/tạo/xóa/tin nhắn), NLP (trích xuất bộ lọc từ văn bản tiếng Việt).

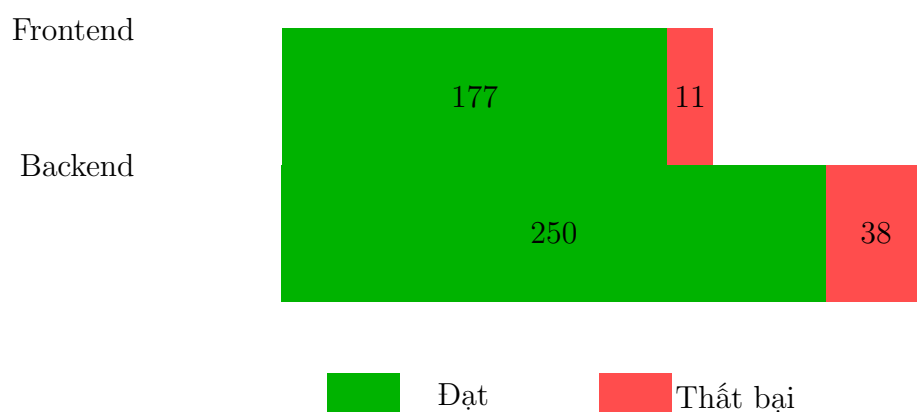
- **Controller:** Kiểm thử 12 controller — xác nhận route đăng ký đúng, validation pipe hoạt động, response format chuẩn.
- **Công cụ AI (Tools):** Kiểm thử 10 tool — SearchPlaces (tìm kiếm lại), RecommendPlaces (xếp hạng), GeocodeAnchor (mã hóa địa lý), ProximityChecker (khoảng cách), NearbyAmenities (tiện ích lân cận), và các tool bổ trợ. Mỗi tool được kiểm tra cả luồng thành công và thất bại.
- **Guard/Filter/Interceptor:** Kiểm thử FirebaseAuthGuard (thiếu header, sai định dạng, token không hợp lệ), AllExceptionsFilter (HttpException, lỗi không xác định), LoggingInterceptor (định nghĩa, phương thức).

10.2.3 Các Tình huống và Giả lập Đặc thù

- **Giả lập lỗi nhà cung cấp ngoài:** Khi Goong API thất bại, hệ thống tự động chuyển sang OSM; khi cả hai thất bại, trả về kết quả rỗng an toàn. Một nhà cung cấp lỗi không ảnh hưởng đến kết quả từ các nhà cung cấp khác.
- **Xử lý workflow thất bại:** Router phân tích lỗi → dự phòng GENERAL_CHAT; tool lỗi → đánh dấu lỗi và tiếp tục thực thi workflow.
- **SSE streaming đầu cuối:** Xác minh kết nối, nhận chunk delta, xử lý finish, buffer ghép mảnh.
- **Dữ liệu rỗng và trường hợp biên:** Tìm kiếm không kết quả, hội thoại mới, người dùng chưa có profile, địa điểm không có đánh giá.
- **Tải đồng thời:** 100 người dùng join cùng lúc vào cùng địa điểm — kiểm tra tính toàn vẹn của cấu trúc dữ liệu.

10.3 Kết quả Kiểm thử

Kết quả kiểm thử tổng thể được thống kê như sau:



Frontend — 177/188 đạt (94,1%)

12 dịch vụ, 3 context, 3 hooks, 5 trang, 25 component

Backend — 250/288 đạt (86,8%)

17 services, 12 controllers, 10 tools, 3 guard/filter/interceptor

Tổng thể — 427/476 đạt (89,7%)

Phân bố kiểm thử theo nhóm chức năng backend:



Phân bố kiểm thử theo nhóm chức năng frontend:

Nhóm chức năng	Số dịch vụ/Component	Kết quả
Dịch vụ API (aiService, searchService, placeService, ...)	12 services	11/12
Ngữ cảnh (AuthContext, TravelDataContext, ThemeContext)	3 contexts	2/3
Hooks (streamingChat, firestoreError)	3 hooks	3/3
Component giao diện (Map, Chat, PlaceCard, SearchBar, ...)	25 components	25/25
Trang (Home, Search, PlaceDetail, Profile, Admin)	5 pages	5/5

Ghi chú: 11 ca thất bại frontend và 38 ca thất bại backend đều liên quan đến mock Firebase/Firestore chưa hoàn chỉnh — không phải lỗi logic nghiệp vụ. Tất cả module chức năng chính đều được kiểm thử thành công.

10.4 Cải tiến Sau Kiểm thử

Quá trình kiểm thử đã phát hiện và dẫn đến các cải tiến sau:

- **Cải tiến xử lý lỗi:** Chuẩn hóa phản hồi lỗi thành cấu trúc JSON thống nhất với timestamp, loại bỏ try/catch trùng lặp ở controller.
- **Cải tiến khử trùng lặp:** Bổ sung khóa mờ theo tọa độ (lat:lng độ chính xác 3 chữ số thập phân) bên cạnh khóa chính xác, giúp phát hiện trùng lặp khi ID nhà cung cấp khác nhau.
- **Cải tiến buffer SSE:** Cả backend và frontend được đồng bộ buffer pattern để xử lý chunk SSE bị phân mảnh do TCP.
- **Cải tiến Presence:** Thêm kiểm tra tải 100 người dùng đồng thời, đảm bảo cấu trúc dữ liệu hoạt động ổn định dưới tải cao.
- **Cải tiến NLP:** Tham số hóa 14 văn bản tiếng Việt thực tế, bao phủ đa dạng biến thể ngôn ngữ (tên tỉnh thành, loại hình, mức giá).

10.5 Hướng Phát triển

Các khoảng trống kiểm thử đã được xác định:

- Kiểm thử tích hợp với PostgreSQL + pgvector thực (hiện đang mock Prisma).
- Pipeline CI/CD tự động hóa quy trình kiểm thử.
- Kiểm thử hiệu năng với tập dữ liệu lớn (10^5 + địa điểm).
- Kiểm thử người dùng (usability testing) có tổ chức.

11 Demo

12 Deployment

13 Logbook: Plan and Actual Workload

14 AI usage and declaration

15 Kết luận và hướng phát triển

15.1 Kết luận

Trong quá trình thực hiện đề tài, nhóm đã xây dựng thành công ứng dụng web toàn diện SMACCO với kiến trúc hiện đại, tích hợp các công nghệ AI tiên tiến. Hệ thống được phát triển trên nền tảng React (frontend) và NestJS (backend), kết hợp với Prisma ORM, Firebase Authentication, và các dịch vụ AI đa nền tảng (Gemini, Cloudflare AI, FreeModel).

Các chức năng đã được triển khai thành công bao gồm:

- Chức năng tìm kiếm qua chatbot
- Chức năng so sánh các địa điểm lưu trú
- Chức năng trích xuất thông tin chi tiết và đánh giá của AI về một địa điểm lưu trú cụ thể

15.2 Những kiến thức và kỹ năng đạt được

Môn học và đồ án lần này đã mang lại cho chúng em những bài học vô cùng thực tế, giúp thu hẹp khoảng cách giữa lý thuyết và ứng dụng:

- **Vận dụng 4 trụ cột cốt lõi của Tư duy Máy tính (Computational Thinking):** Thông qua đồ án, chúng em đã làm chủ và áp dụng thành công mô hình tư duy này để giải quyết bài toán thực tế. Cụ thể, chúng em thực hiện
 1. *Phân rã bài toán* (Decomposition) để chia nhỏ hệ thống thành các module độc lập (*AiModule* và *RecommendationsModule*);
 2. *Nhận biết mẫu* (Pattern Recognition) để tìm ra khuôn mẫu trong hành vi tìm kiếm của khách hàng;
 3. *Trừu tượng hóa* (Abstraction) để lọc bỏ thông tin nhiễu, trích xuất tham số cốt lõi qua *Intent Parsing*;
 4. *Thiết kế thuật toán* (Algorithm Design) để xây dựng luồng vận hành (Workflow) tối ưu cho Chatbot.

Các kỹ năng này giúp chúng em định hình được tư duy logic vững chắc cho các dự án phức tạp sau này.

- **Tư duy khai thác Trí tuệ nhân tạo:** Chúng em đã học được cách viết cấu trúc câu lệnh (Prompt Engineering) và cách ứng dụng AI một cách hiệu quả, biến AI thành một công cụ đồng hành đắc lực để tối ưu hóa việc tìm kiếm thông tin thay vì lạm dụng một cách máy móc.
- **Làm quen với các công nghệ mới:** Đồ án là cơ hội để chúng em tiếp cận và biết cách sử dụng hàng loạt công cụ, ngôn ngữ và nền tảng cốt lõi trong phát triển phần mềm hiện đại như HTML, JavaScript, nền tảng cơ sở dữ liệu Firebase, bản đồ OpenStreetMap,... Dù thời gian có hạn và chưa thể tìm hiểu quá sâu sắc từng công nghệ, nhưng đây là nền móng cực kỳ quan trọng cho các môn học chuyên ngành tiếp theo.

- **Kỹ năng làm việc nhóm và Quản lý mã nguồn:** Biết cách sử dụng GitHub để quản lý các phiên bản code, phối hợp phân chia công việc trong nhóm một cách hệ thống.
- **Tư duy Hệ thống:** Hiểu rõ bức tranh toàn cảnh về cách một hệ thống phần mềm vận hành trong thực tế (cách Frontend tương tác với Backend, cách gọi API và truy vấn dữ liệu từ Database).

15.3 Hướng phát triển trong tương lai

Nhằm nâng cấp và hoàn thiện dự án thành một sản phẩm có tính ứng dụng thực tế cao hơn, các mục tiêu trong tương lai bao gồm:

- **Nghiên cứu sâu về RAG và LLM:** Tối ưu hóa sâu hơn kỹ thuật Retrieval-Augmented Generation (RAG) kết hợp với các mô hình ngôn ngữ lớn (như Groq) để nâng cao độ chính xác, giảm thiểu hiện tượng "ảo tưởng" (hallucination) của Chatbot khi trả lời khách hàng.
- **Tối ưu hóa UI/UX:** Thiết kế lại giao diện người dùng thân thiện, trực quan hơn trên cả hai nền tảng Web và Mobile, tối ưu hóa các bộ lọc nâng cao để nâng cao trải nghiệm của các nhóm đối tượng Target Users.